

Intelligent Cognitive Alarm Platform

Milestone 2 Report

Wake-Up Verification & Challenge Evaluation Modules

Prepared by: Komala from team 5

Table of Contents

1. Project Description.....	3
1.1 Module Scope	3
1.2 Captured Data	3
2. Environment Setup	3
2.1 Tech Stack	3
3. Data Model.....	3
3.1 Wake-Up Verification.....	3
3.2 Challenge Evaluation	4
4. Proposed System	4
4.1 Verification Flow.....	4
4.2 Evaluation Flow	4
5. Implementation Pipeline	5
5.1 Wake-Up Verification.....	5
5.2 Challenge Evaluation	5
6. Verification & Testing	5
7. API Implementation — Swagger Documentation.....	5
7.1 GET /verification/ — Retrieve All Verifications.....	5
7.2 POST /verification/ — Create Verification	6
7.3 DELETE /verification/{verification_id} — Delete Verification	7
7.4 GET /evaluation/ — Retrieve All Evaluations.....	7
7.5 POST /evaluation/ — Create Evaluation	8
7.6 Endpoint Summary	8
8. Conclusion	9

1. Project Description

The Intelligent Cognitive Alarm Platform is an AI-powered system that helps users build consistent wake-up habits by requiring them to solve personalized cognitive challenges before an alarm can be dismissed. Instead of allowing a simple snooze or dismiss action, the platform requires the user to be mentally active by correctly answering a challenge, with performance data feeding into the platform's broader habit-tracking and difficulty-adaptation systems.

This report covers two backend modules delivered in Milestone 2: Wake-Up Verification, which checks whether a user's submitted answer is correct, and Challenge Evaluation, which records the user's overall performance on a completed challenge.

1.1 Module Scope

- Wake-Up Verification — validating a user's submitted answer against the correct answer and recording the outcome
- Challenge Evaluation — recording score, attempts, time taken, and pass/fail status once a challenge is completed

1.2 Captured Data

- User answer and correct answer for each verification attempt
- Whether the submitted answer was correct
- Score, attempt count, and time taken for each evaluated challenge
- Pass/fail status for each completed challenge

2. Environment Setup

Both modules were developed as backend services within the platform's microservices layer, exposed through FastAPI and backed by a PostgreSQL database. All endpoints were implemented, tested, and documented using FastAPI's auto-generated Swagger UI.

2.1 Tech Stack

Layer	Technology
Backend Framework	Python, FastAPI
Primary Database	PostgreSQL
ORM	SQLAlchemy
Schema Validation	Pydantic
API Testing	Swagger UI (/docs)

3. Data Model

The two modules are modeled as separate entities, each persisted independently in PostgreSQL.

3.1 Wake-Up Verification

Field	Type	Description
id	integer	Verification record identifier
user_id	integer	User who attempted the challenge
challenge_id	integer	Challenge being verified
user_answer	string	Answer submitted by the user
correct_answer	string	Expected correct answer
is_correct	boolean	Result of the comparison
verified_at	datetime	Timestamp of verification

3.2 Challenge Evaluation

Field	Type	Description
id	integer	Evaluation record identifier
user_id	integer	User being evaluated
challenge_id	integer	Challenge that was completed
score	integer	Score achieved on the challenge
attempts	integer	Number of attempts taken
time_taken	integer	Time taken, in seconds
status	string	Pass/fail outcome
evaluated_at	datetime	Timestamp of evaluation

4. Proposed System

Both modules follow the platform's standard request pipeline: a request is received by the FastAPI router, handled by a service layer containing the business logic, and persisted through SQLAlchemy ORM to the PostgreSQL database, which then returns a structured response.

4.1 Verification Flow

- User submits an answer for a challenge
- The Verification API receives the request
- The submitted answer is compared against the correct answer
- The result (correct or incorrect) is stored with a timestamp

4.2 Evaluation Flow

- A challenge is marked as completed
- The Evaluation API receives the completion data
- Score, attempts, time taken, and status are stored against the user and challenge

5. Implementation Pipeline

Milestone 2 focused on implementing both modules end-to-end at the backend: routes, schema, validation, and persistence.

5.1 Wake-Up Verification

- FastAPI routes to create, retrieve, and delete verification records
- PostgreSQL schema storing user answer, correct answer, and comparison result
- Request/response validation using Pydantic models

5.2 Challenge Evaluation

- FastAPI routes to create, retrieve, and delete evaluation records
- PostgreSQL schema storing score, attempts, time taken, and status
- Request/response validation using Pydantic models

6. Verification & Testing

Both modules were verified against the Milestone 2 evaluation criteria: create, retrieve, and delete operations were exercised directly through the FastAPI Swagger UI, with live request and response payloads captured for each endpoint.

- Endpoint-level testing (create, read, delete) for both modules
- Schema validation for required fields (user answer, correct answer, score, attempts, status)
- Response payload verification against the expected schema for each endpoint
- Manual testing through the Swagger UI (/docs) with live server responses

7. API Implementation — Swagger Documentation

The Wake-Up Verification and Challenge Evaluation endpoints were implemented in FastAPI and verified directly against the auto-generated Swagger UI (/docs). The core operations for each module are shown below with live request/response payloads.

7.1 GET /verification/ — Retrieve All Verifications

Returns the stored verification records, including the user's answer, the correct answer, and whether the submission was correct.

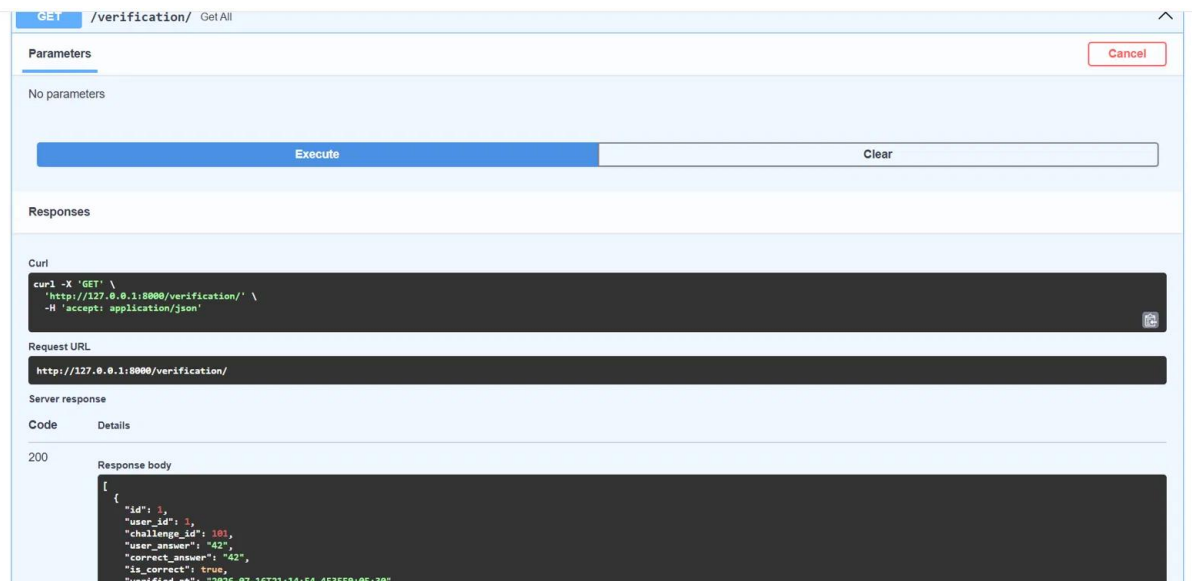


Figure 1: Swagger UI test of GET /verification/ — Get All.

7.2 POST /verification/ — Create Verification

Accepts a new verification payload (user ID, challenge ID, user answer, correct answer) and persists it, returning the created record with the comparison result and timestamp.

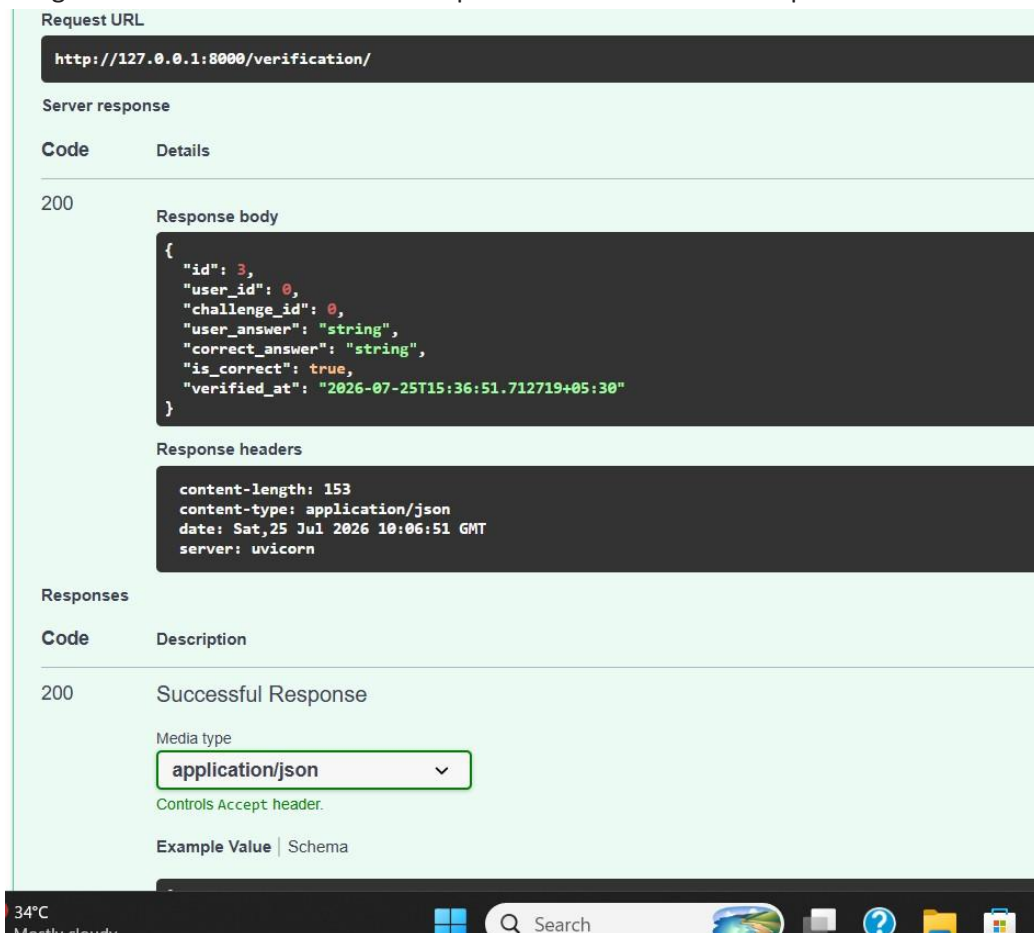


Figure 2: Swagger UI test of POST /verification/ — Create.

7.3 DELETE /verification/{verification_id} — Delete Verification

Deletes a verification record by ID, returning a confirmation message on success.

The image shows a Swagger UI interface for testing the DELETE /verification/{verification_id} endpoint. The 'verification_id' parameter is set to 1. The 'Execute' button is highlighted in blue. Below the button, the 'Responses' section shows the results of the test. The 'Curl' section displays the command: `curl -X 'DELETE' \ 'http://127.0.0.1:8000/verification/1' \ -H 'accept: application/json'`. The 'Request URL' section shows: `http://127.0.0.1:8000/verification/1`. The 'Server response' section shows a 200 status code. The 'Response body' section displays the JSON response: `{ "message": "Verification deleted successfully" }`. The 'Response headers' section displays the following headers: `content-length: 47`, `content-type: application/json`, `date: Sat, 25 Jul 2026 10:07:24 GMT`, and `server: uvicorn`.

Code	Details
200	Response body <pre>{ "message": "Verification deleted successfully" }</pre> Response headers <pre>content-length: 47 content-type: application/json date: Sat, 25 Jul 2026 10:07:24 GMT server: uvicorn</pre>

Figure 3: Swagger UI test of DELETE /verification/{verification_id}.

7.4 GET /evaluation/ — Retrieve All Evaluations

Returns the stored evaluation records, including score, attempts, time taken, and pass/fail status for each completed challenge.



Figure 4: Swagger UI test of GET /evaluation/ — Get All.

7.5 POST /evaluation/ — Create Evaluation

Accepts a new evaluation payload (user ID, challenge ID, score, attempts, time taken, status) and persists it, returning the created record with a timestamp.



Figure 5: Swagger UI test of POST /evaluation/ — Create.

7.6 Endpoint Summary

Module	Method	Path	Status
Wake-Up Verification	GET	/verification/	Working
Wake-Up Verification	POST	/verification/	Working

Wake-Up Verification	GET	/verification/{id}	Working
Wake-Up Verification	DELETE	/verification/{id}	Working
Challenge Evaluation	GET	/evaluation/	Working
Challenge Evaluation	POST	/evaluation/	Working
Challenge Evaluation	GET	/evaluation/{id}	Working
Challenge Evaluation	DELETE	/evaluation/{id}	Working

8. Conclusion

Milestone 2 establishes the Wake-Up Verification and Challenge Evaluation modules of the Intelligent Cognitive Alarm Platform: validating user answers against expected challenge answers, and recording score, attempts, time taken, and pass/fail status for each completed challenge, through a FastAPI/PostgreSQL backend. With these endpoints in place and verified through the Swagger UI, the platform is positioned to integrate this verification and scoring data into the Adaptive Difficulty Engine and Habit Scoring Engine in subsequent milestones.